

前置知识一：数据流模型

1. 数据流模型概述

1.1 定义

数据流模型 (Data Stream Model) 是一种用于处理**连续、大规模、快速到达**的数据序列的计算模型。

- **传统数据库模型假设**：数据已经存储在磁盘中，计算任务可以多次扫描数据。
- **数据流模型假设**：数据不断产生，以流 (Stream) 的形式到达，算法只能**顺序读取一次或少数几次**，并且不能保存全部数据。

形式化地表示，一个数据流可以定义为：

$$S = \{a_1, a_2, a_3, \dots, a_n\}$$

其中：

- a_i ：第 i 个到达的数据元素
- n ：数据流长度

数据按照时间顺序不断进入系统：

$$a_1 \rightarrow a_2 \rightarrow \dots \rightarrow a_n$$

算法接收到数据后，需要维护一个较小的状态：

$$M_t = f(a_1, a_2, \dots, a_t)$$

其中：

- M_t ：当前时刻保存的状态
- f ：数据流算法
- t ：当前已经处理的数据数量

最终通过保存的小规模状态恢复最终结果：

$$\text{Answer} = g(M_n)$$

2. 为什么需要数据流模型？

2.1 传统模型的问题

随着互联网的发展，数据规模快速增长，例如：

- 社交网络消息
- 网络访问日志
- 传感器数据
- 金融交易记录
- 用户点击行为

数据量达到 $10^{12} \sim 10^{15}$ 级别。

存储估算：假设每条数据占 100 Byte，数据量为 10^{12} ，则：

$$\text{Storage} = 100 \times 10^{12} \text{ Byte} = 100 \text{ TB}$$

普通计算机无法保存。

2.2 数据流模型的核心思想

用少量内存保存**数据摘要 (summary)**，而不是保存全部数据。

即对原始数据 $D = (a_1, a_2, \dots, a_n)$ 进行转换：

$$D \rightarrow M, \quad \text{其中 } |M| \ll |D|$$

3. 数据流模型数学描述

3.1 数据流计算框架

数据流算法通常包含三个部分：

1. 输入流

$$S = a_1, a_2, \dots, a_n$$

2. 状态更新函数：每当新数据到达时

$$M_i = \text{Update}(M_{i-1}, a_i)$$

3. **查询函数**：最终输出

$$\text{Result} = \text{Query}(M_n)$$

整个过程为状态的不断演化：

$$M_0 \rightarrow M_1 \rightarrow \dots \rightarrow M_n$$

4. 数据流模型的重要限制

4.1 单遍扫描 (Single Pass)

算法只能将 $a_1, a_2, \dots, a_n$ 按顺序访问一次，不能多次执行 $\text{Scan}(S)$ 。

原因：数据规模巨大，重新读取成本太高。

4.2 有限内存 (Limited Memory)

模型	空间复杂度
传统算法	$\text{Memory} = O(n)$
数据流算法	$\text{Memory} = O(\text{poly}(\log n))$

举个栗子：统计 10 亿个数字

- 传统方法：保存 10^9 个元素
- 数据流方法：只保存 $O(\log n)$ 个计数器

5. 典型数据流算法

5.1 频率估计算法 Count-Min Sketch

5.1.1 问题

给定数据流 $S = a_1, a_2, \dots, a_n$ ，要求查询元素 x 出现的次数 f_x 。

5.1.2 直接方法

建立哈希表：

word -> count

空间复杂度 $O(n)$ ，缺点是数据太大时不可行。

5.1.3 Count-Min Sketch 思想

建立二维数组 $C[d][w]$ ，其中：

- d ：哈希函数数量
- w ：桶数量

更新：当元素 x 到来时，对每个哈希函数 h_i ：

$$C[i][h_i(x)] += 1$$

查询：取所有行中的最小值作为估计：

$$\hat{f}_x = \min_i C[i][h_i(x)]$$

5.1.4 误差分析

Count-Min Sketch 保证：

$$\Pr \left[\hat{f}_x \leq f_x + \epsilon N \right] \geq 1 - \delta$$

其中：

- N ：数据流总长度
- ϵ ：误差参数
- δ ：失败概率

空间复杂度：

$$O \left(\frac{1}{\epsilon} \log \frac{1}{\delta} \right)$$

5.2 Bloom Filter (布隆过滤器)

5.2.1 问题

判断一个元素是否出现过。

例子：网站判断用户是否已注册。

5.2.2 数据结构

Bloom Filter 包含：

- m 个位的数组 $B[1 \dots m]$
- k 个哈希函数 $h_1, h_2, \dots, h_k$

插入元素 x ：

$$B[h_i(x)] = 1 \quad (i = 1, 2, \dots, k)$$

查询元素 x ：

- 如果 $\exists i, B[h_i(x)] = 0$, 则 x **一定不存在**
- 否则, x **可能存在**

5.2.3 数学分析

误判概率（假阳性率）：

$$P = \left(1 - e^{-kn/m}\right)^k$$

最佳哈希函数数量：

$$k = \frac{m}{n} \ln 2$$

5.3 HyperLogLog

5.3.1 问题

估计**不同元素的数量**（基数估计）。

举个栗子：统计一天有多少用户访问网站。

精确方法：保存集合 $\{user_1, user_2, \dots\}$, 空间 $O(n)$ 。

5.3.2 核心思想

利用随机哈希 $h(x)$ ，观察二进制表示的**前导 0 数量** $\rho(x)$ 。

栗子： 00101101 的前导 0 数量为 2。

若观察到的最大前导 0 数量为 R ，则基数估计为：

$$N \approx 2^R$$

6. 数据流模型中的随机化思想

为什么数据流算法大量使用随机算法？

因为无法保存全部数据，因此利用随机采样 $\text{Sample}(S)$ 来估计 $\text{Query}(S)$ 。

6.1 Reservoir Sampling（蓄水池采样）

目标： 从 n 个数据中随机选择 k 个。

算法： 第 i 个元素以概率 $\frac{k}{i}$ 被选中（替换蓄水池中的元素）。

最终： 每个元素被选中的概率均为

$$P = \frac{k}{n}$$

7. 数据流模型应用

7.1 网络安全

场景： 网络攻击检测。网络流为连续的 IP 访问序列。

问题： 寻找高频访问 IP、异常流量。

方法： 使用 Count-Min Sketch 统计 $\text{frequency}(\text{IP})$ 。

7.2 搜索引擎

场景：Google 搜索日志每天约有 10^{10} 次查询。

问题：热门关键词统计。

方法：

- Bloom Filter：判断关键词是否出现过
- Count-Min Sketch：统计 keyword frequency

7.3 推荐系统

用户行为流： (user, item, time)

栗子：用户依次点击"手机 → 电脑 → AI课程"，需要实时更新用户兴趣。

方法：数据流模型实时维护 Preference(user)。

7.4 金融交易监控

交易流： $T_1, T_2, \dots, T_n$

问题：检测异常交易。

栗子：正常交易 amount < 10000，突然出现 amount = 10000000 → 触发风险预警。

8. 数据流模型与传统数据库比较

特点	数据库模型	数据流模型
数据状态	静态	动态
访问方式	随机访问	顺序访问
存储	完整保存	保存摘要
计算次数	多次扫描	一次扫描
空间	$O(n)$	$O(\log n)$
结果	精确	近似

9. 数据流算法误差理论

由于采用摘要，结果通常是估计值 $\hat{x}$ 而非精确值 x ，需要分析误差：

1. 加性误差

$$|\hat{x} - x| \leq \epsilon$$

2. 相对误差

$$\frac{|\hat{x} - x|}{x} \leq \epsilon$$

3. 概率保证：通常要求

$$\Pr(\text{Error} > \epsilon) < \delta$$

即算法以概率 $1 - \delta$ 保证误差在 ϵ 以内。

10. 数据流模型的优点

1. **高空间效率**：空间从 $O(n)$ 降至 $O(\log n)$
2. **实时处理**：数据到达后立即更新 $M_i = \text{Update}(M_{i-1}, a_i)$
3. **可扩展性强**：适合 TB / PB 级数据

11. 数据流模型的缺点

1. **结果通常近似**：Data $\rightarrow$ Summary 存在信息损失
2. **算法设计困难**：需要数学证明正确率、空间复杂度、时间复杂度
3. **不适合复杂查询**：例如多表 Join 较困难